

StudyHub API

Backend Project Requirements Document

Project Overview

- Build a backend API for an online learning platform called StudyHub.
- The platform allows teachers to create courses, students to enroll, and admins to manage the system.
- Use Express, MongoDB, Mongoose, JWT, bcrypt, and MVC architecture.

Technical Requirements

- Express.js
- MongoDB + Mongoose
- JWT Authentication
- bcrypt Password Hashing
- MVC Architecture
- Environment Variables (.env)
- Middleware-Based Authorization

Folder Structure

- src/config
- src/controllers
- src/middleware
- src/models
- src/routes
- src/app.js
- src/server.js

User Model

- name (required)
- email (required, unique)
- password (required, hashed)
- role (admin, teacher, student)
- createdAt

Course Model

- title

- description
- category
- teacher (ObjectId reference to User)
- createdAt

Enrollment Model

- student (ObjectId reference to User)
- course (ObjectId reference to Course)
- enrolledAt

Authentication Features

- POST /api/auth/register
- POST /api/auth/login
- Hash passwords before saving
- Generate JWT on successful login
- Protect private routes using middleware

Authorization Rules

- Student: view courses and enroll
- Teacher: create and update own courses
- Admin: full access to all resources

Course Routes

- POST /api/courses (Teacher/Admin only)
- GET /api/courses (Public)
- GET /api/courses/:id (Public)
- PATCH /api/courses/:id (Owner Teacher/Admin)
- DELETE /api/courses/:id (Admin only)

Enrollment Routes

- POST /api/enrollments (Student only)
- GET /api/enrollments/my (Authenticated Student)

Search, Filtering & Pagination

- /api/courses?search=node
- /api/courses?category=backend
- /api/courses?page=1&limit;=10

Learning Goals

- JWT authentication flow
- Role-based authorization
- MongoDB relationships
- populate()
- Pagination
- Filtering
- Search
- Professional MVC architecture

Submission Checklist

- Complete folder structure
- Authentication implemented
- Authorization implemented
- Relationships implemented using populate()
- Pagination implemented
- Filtering implemented
- Search implemented
- Ready for code review